Mastering Claude Code: The Complete Architecture-Informed Guide

> Everything a software engineer needs to know, grounded in how Claude Code actually works under the hood — from model training to production workflow.

---

1. Understand What You're Actually Working With

Claude Code is **not** an autocomplete tool or a chat wrapper. It's a single-threaded agentic loop (internally codenamed "nO") that calls the Claude model repeatedly, executing tools on your local machine between each call. Every time the model returns `tool_use` blocks, the loop executes them (file reads, bash commands, edits) and feeds results back. When the model returns plain text with no tool calls, the loop ends and control returns to you.

The key mental model: Your prompt doesn't just generate code — it launches an autonomous agent that will read, think, act, verify, and iterate. The quality of that entire multi-step loop depends on how well you set up its context, how clearly you define the task, and how effectively you constrain the action space.

What happens on every API call: Claude Code assembles a prompt from your system prompt, CLAUDE.md contents, tool definitions (including any MCP servers), conversation history, and your latest message. This goes to the Anthropic API where it still passes through auth, input moderation, inference, output moderation, and streaming — Claude Code doesn't bypass those safety layers. But Claude Code *controls* the system prompt, tool orchestration, and context management layers, running them locally instead of server-side.

The tools Claude has access to:

- **Read tools:** Read, Glob, Grep, LS, NotebookRead — for understanding your codebase
- **Write tools:** Edit, MultiEdit, Write, NotebookEdit — for making changes
- **Execution:** Bash (persistent shell sessions) — for running anything: tests, builds, git, docker
- **Network:** WebFetch, WebSearch — for documentation and research
- **Planning:** TodoRead, TodoWrite — for structured task tracking
- **Delegation:** Task (subagents) — for parallel or isolated work

Understanding this architecture explains *why* every best practice works. Nothing in this guide is arbitrary — it all flows from the mechanics of the system.

---

2. Understand How the Model Was Trained — It Predicts Your Results

Claude was trained in four stages, and understanding each stage helps you predict exactly when it will excel and when it will fail.

Stage 1 — General Pre-training: Next-token prediction on trillions of tokens (web, books, code, papers). This gives Claude broad knowledge of language and code patterns across all major languages and frameworks. It

learns what code *looks like*, not what code *does*.

Stage 2 — Code-focused Pre-training: Continued training on curated, high-quality code corpora — full repositories, commit diffs, pull request descriptions, issue-to-patch pairs. This deepens code-specific patterns and teaches repository-level understanding (file relationships, imports, cross-file dependencies).

Stage 3 — Supervised Fine-tuning (SFT): Training on human-written (instruction, response) pairs. This teaches the model to follow instructions, use tools effectively, and behave as a helpful coding assistant. It learns *format and style* of being helpful.

Stage 4 — Reinforcement Learning (RLHF + RLVR): The critical differentiator. The model generates code, it's executed against test cases, and pass/fail becomes the reward signal. This teaches the model to produce code that actually *works*, not just code that *looks right*. It's where self-verification, backtracking, multi-step reasoning, and error recovery emerge.

Why this matters for your daily work

The gap between "looks right" and "is right" is the fundamental challenge. Pre-training teaches textual patterns; RL teaches execution correctness. But the RL bridge is imperfect — the model still defaults to pattern matching when it hasn't been sufficiently trained on a specific type of problem. This is why:

- **Verification is non-optional** — the model can produce plausible but incorrect code
- **Popular patterns are more reliable** — more training data = stronger statistical prior
- **Novel problems need more human oversight** — weak prior = higher error rate
- **Types and constraints help** — they narrow the output space toward correct solutions

---

3. Know Where the Model Is Strong vs Weak

The model's capability at any task is predictable from three factors: **training data density** (how much example code exists), **pattern regularity** (how template-like the task is), and **RL verifiability** (whether it could be tested during training).

Tasks where Claude excels

- **Boilerplate and CRUD** — highly repetitive, template-based, millions of examples
- **Front-end development (React, Vue, Angular)** — massive training data, visual and template-based patterns, type-constrained with TypeScript
- **REST/GraphQL APIs** — extremely common patterns with overwhelming statistical signal
- **Test generation** — millions of test files in training data, strong assertion patterns
- **Refactoring** — common refactoring patterns are well-documented
- **Documentation and explanation** — the model is literally a language model; explaining things is its core competency
- **Regex, configs, type definitions** — highly structured, deterministic output

Tasks where Claude is moderate

- **Bug fixing** — strong for common patterns, weaker for subtle business logic errors
- **Performance optimization** — knows common patterns, struggles with profiling-driven tuning
- **Concurrency** — knows async/await at a pattern level, unreliable for race conditions and deadlocks
- **Code review** — catches common issues, misses subtle architectural concerns

Tasks where Claude struggles

- **Novel algorithm design** — no statistical prior for truly new algorithms
- **Security-critical code** — has seen both secure and insecure patterns, can't reliably distinguish. Always security review
- **Complex distributed systems** — deep failure-mode reasoning with limited training examples
- **Legacy systems (COBOL, mainframes)** — minimal training data, high hallucination risk
- **Exact computation** — token prediction is not arithmetic. Delegate to code execution

Language and framework strength follows training data density

Strongest: Python, TypeScript/JavaScript, Java, Go, Rust, SQL — massive GitHub and documentation presence

Good: C#, Ruby (especially Rails), Swift, Kotlin, PHP (but tends toward older patterns)

Unreliable: COBOL, Fortran, Ada, Assembly, APL — minimal training data, high hallucination risk

Framework rule of thumb: The more mainstream the framework, the better Claude performs. React > Svelte > Qwik. Express > Hono > Fresh. Django > FastAPI > Litestar. Newer or niche frameworks have less training data.

Practical implications

- **Use TypeScript over JavaScript** — type annotations constrain the output space, dramatically improving generation quality. Types are prompts embedded in code.
- **For weak areas, compensate with context** — provide documentation, examples, and API references in the context window. The less training data, the more context *you* must supply.
- **For novel problems, split the work** — write the novel logic yourself, have Claude write the boilerplate, tests, and infrastructure around it.
- **For security, never trust alone** — always run SAST/DAST tools and expert review on auth, encryption, and access control code.
- **For rapidly-changing APIs, always fetch current docs** — training data is stale. Tell Claude to WebFetch the current API documentation before implementing.

4. CLAUDE.md Is Your Highest-Leverage Investment

Why it matters architecturally

CLAUDE.md is loaded into **every single session** as part of the system prompt. It's the only persistent memory Claude Code has between sessions. The model is stateless — its weights are frozen, it learns nothing between conversations. CLAUDE.md is how you bridge that gap.

Think of it as onboarding documentation for a brilliant but amnesiac contractor who joins your team fresh every morning. They can learn fast, but they start from zero every time.

What to include: The WHAT, WHY, HOW framework

WHAT — Give Claude a map of your codebase:

- Project structure and tech stack
- Key directories and what they contain
- Monorepo layout (which apps, shared packages, what everything is for)
- Database schema locations and ORM patterns

WHY — Explain the purpose and rationale:

- Purpose of each major module
- Architectural decisions and *why* they were made (not just what they are)
- Design patterns in use and why they were chosen
- Known quirks, gotchas, or legacy decisions

HOW — Tell Claude how to work on the project:

- Build, test, lint, typecheck commands
- How to verify changes (which test suite, which linting config)
- Package manager (bun vs npm vs pnpm)
- Branch naming, commit conventions, PR process
- Environment setup notes (pyenv, nvm, docker requirements)

What NOT to include

- Instructions Claude already follows correctly without being told (wasted context)
- Rare or situation-specific instructions (use Skills — they load on demand)
- Negative-only rules like "never use X" — always provide the alternative: "never use X; prefer Y instead"

- Entire API docs or reference material — instead, reference the file path: "For MUI patterns, see `src/components/README.md`"

Compensate for training data gaps in CLAUDE.md

If your project uses niche frameworks or languages where Claude has weaker training data, CLAUDE.md becomes even more critical. Include:

- Links to documentation Claude should fetch before implementing
- Code examples of your preferred patterns (few-shot examples create overwhelming signal)
- Explicit style instructions for languages where Claude defaults to stale patterns (e.g., "Always use modern PHP 8.2+ features: enums, readonly properties, fibers")

Keep it lean — this is critical

Research suggests frontier thinking models can reliably follow approximately 150-200 instructions. Claude Code's own system prompt already consumes about 50 of those. Target: under ~500 lines. Ruthlessly prune. Version control it.

Use hierarchical CLAUDE.md files

Put general project rules in the root, specific rules in subdirectories (`/frontend/CLAUDE.md`, `/backend/CLAUDE.md`). Claude prioritizes the most specific (most nested) file when working in that area. This is especially powerful in monorepos.

---

5. Plan Before You Build — Every Time

Why planning matters architecturally

Each step in an agentic chain introduces error probability. If each step is 95% accurate, 10 steps give you only ~60% end-to-end accuracy. Planning reduces the total number of steps by getting direction right upfront. This is even more critical for tasks in Claude's "moderate" or "weak" zones — where each step has a higher base error rate.

The interview pattern (for larger features)

```
I want to build [brief description]. Interview me in detail using
the AskUserQuestion tool. Ask about technical implementation, UI/UX,
edge cases, concerns, and tradeoffs. Don't ask obvious questions —
dig into the hard parts I might not have considered. Keep interviewing
until we've covered everything, then write a complete spec to SPEC.md.
```

Then start a **fresh session** (`/clear`) to implement the spec.

The spec-then-execute pattern

1. Session 1: Research, plan, write to `PLAN.md` or `SPEC.md`
2. You review and refine the plan
3. Session 2 (fresh): "Implement the plan in PLAN.md step by step"

Plan Mode (Shift+Tab twice)

Puts Claude in architect mode — can observe and analyze but cannot modify files. Use it to get analysis before you let it act.

Use TodoWrite for structured execution

```
Before writing any code, create a detailed TODO plan with at least
10 items. Mark each with priority. Complete items one at a time,
running tests after each change. Mark items complete as you go.
```

Match planning depth to task difficulty

- **Claude's strong zone** (boilerplate, CRUD, standard patterns): Light planning, let Claude run
- **Claude's moderate zone** (bug fixing, optimization, integration): Plan first, verify each step
- **Claude's weak zone** (novel algorithms, concurrency, security): Plan extensively, write specs, human-review each change

---

6. Context Management Is the Core Skill

Why this is the #1 skill architecturally

The context window is Claude's entire working memory. It holds system prompt, CLAUDE.md, MCP tool definitions, conversation history, file contents, and command outputs. When it fills up (around 70% capacity), auto-compaction triggers. Every token is processed on every API call — bloated context degrades quality, increases latency, and costs more.

Start fresh sessions aggressively

Use `/clear` between unrelated tasks. Use `/rename` before clearing so you can `/resume` later.

Use `/compact` manually at logical breakpoints

Don't wait for auto-compact. Direct the compaction: `/compact Focus on the API changes and test results`.

Scope investigations narrowly

"Investigate the authentication system" reads dozens of files. Instead: "Read `src/auth/login.ts` and

`src/auth/middleware.ts` and explain the JWT validation flow."

Put persistent rules in CLAUDE.md, not conversation

Instructions early in conversation may be lost during compaction. CLAUDE.md survives compaction.

Watch MCP server token costs

Every MCP server's tool definitions load into context at session start. Run `/mcp` to check per-server costs. Disconnect servers you don't need.

The "write to file" pattern for long tasks

Write progress to external files (`PROGRESS.md`, `DECISIONS.md`, `TODO.md`). A fresh session can read these and resume with full signal quality, avoiding "summary of summary" degradation.

---

7. Write Prompts Like Engineering Requirements

Why precision matters architecturally

The model is a probability engine that follows the strongest signal. Ambiguous prompts produce ambiguous results. Specific prompts concentrate probability on the correct interpretation.

Be specific about inputs, outputs, and constraints

```
❌ "Add tests for the auth module"

✅ "Write unit tests for src/auth/login.ts using Jest and React
   Testing Library. Cover: successful login, invalid credentials,
   expired token, and network timeout. Follow the patterns in
   src/auth/signup.test.ts. Mock API calls using src/test/mocks/api.ts."
```

Provide context Claude's training data may lack

- Paste URLs to docs for niche or new APIs (Claude will fetch them)
- Drag-and-drop screenshots for UI tasks
- Pipe data: `cat error.log | claude "explain this error and fix it"`
- For niche frameworks: "Read the Hono.js docs at [URL] before implementing the route handler"

Set explicit constraints

```
Output: TypeScript with strict types
Style: follow existing patterns in src/utils/
Testing: Jest, 80%+ coverage
```

```
Dependencies: don't add new packages without asking
Scope: minimum change necessary — do not refactor
```

The "devil's advocate" pattern before shipping

```
Play devil's advocate on this change:
- What edge cases did I miss?
- What could fail under load?
- What assumptions might be wrong?
Be pessimistic. Think like a hacker, not a builder.
```

---

8. Verification Is Non-Optional

Why this is architecturally critical

The model was trained to always produce helpful, plausible output (RLHF rewarded helpfulness). There's no internal "I'm not sure" signal. Even after RLVR training, the gap between "looks right" and "is right" remains significant. This is especially true for tasks outside the model's strong zone.

Build verification into every interaction

Add to CLAUDE.md:

```
## After ANY code change:
1. Run `npm run typecheck`
2. Run `npm run lint -- --fix`
3. Run `npm run test -- --related`
If any fail, fix them before moving on. Never declare complete
without all three passing.
```

Use hooks for deterministic guarantees

CLAUDE.md rules are advisory. Hooks always execute:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write(*.ts)",
      "hooks": [{ "type": "command", "command": "npx prettier --write \"$file\"" }]
    }]
  }
}
```

Git is your safety net

Work on branches. Commit before large changes. Review diffs. Revert checkpoints when needed.

Increase verification for weak areas

For tasks in Claude's moderate/weak zones, add extra verification:

- **Concurrency code:** Write explicit tests for race conditions
- **Security code:** Run SAST tools as a hook, mandate human security review
- **Niche language code:** Run the compiler/interpreter as part of the verification loop
- **API integrations:** Test against actual (sandboxed) API endpoints, not just mocks

---

9. Model Selection Is an Architectural Decision

Understanding the tradeoffs

Claude Opus 4.6 — Most capable. Best for complex architecture, multi-step reasoning, ambiguous problems. Expensive. Add to CLAUDE.md: "Use subagents only for parallel, independent workstreams. For simple searches, use Grep/Glob directly."

Claude Sonnet 4.5 — The workhorse. Most daily coding, ~80% cheaper than Opus. Your default for implementation.

Claude Haiku 4.5 — Fast and cheap. Simple tasks, quick lookups. Degrades fast with many instructions.

Model selection by task type

- **Architecture and planning:** Opus (complex reasoning justifies the cost)
- **Implementation from specs:** Sonnet (strong patterns, cost-effective)
- **Boilerplate and CRUD:** Sonnet or even Haiku (patterns are so strong that cheaper models handle them)
- **Novel/weak-zone tasks:** Opus with high thinking budget (you need maximum reasoning power)
- **Code review:** Sonnet (good enough for common issues, use Opus for security-critical review)

Extended thinking: powerful but expensive

Thinking tokens are billed as output tokens. For Opus 4.6, use the `effort` parameter: low for simple tasks, high for complex reasoning. Disable thinking for trivial lookups.

Cost-aware subagent model switching

```bash
```

```
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
```

Opus for main reasoning, Sonnet for focused subagent tasks.

---

10. Master Subagents for Context and Parallelism

Each subagent gets its own fresh context window and works independently, returning only a summary. This means exploration doesn't pollute your main context and multiple tasks can run in parallel.

Built-in subagents

- **Explore** — Read-only codebase search
- **Plan** — Research during Plan Mode
- **Task** — Full capabilities, multi-step operations

When to use vs not

Use: Independent parallel tasks, verbose exploration, isolated deep dives. **Don't use:** Simple tasks (Grep is faster), sequential work needing shared context, when you need real-time visibility.

Critical tip: Invocation quality

Most subagent failures are invocation failures — vague instructions sent to the subagent. Include specific file paths, clear success criteria, and relevant context.

---

11. Know the Failure Modes and Their Fixes

Failure Mode	Root Cause	Fix
Correction spiral	Context polluted with wrong attempts	After 2 failed corrections, `/clear` and write a better initial prompt
Infinite exploration	Unbounded investigation fills context	Scope tightly, or use subagents
Plausible hallucination	Statistical pattern match ≠ correctness	Verify with tests, types, compilation. Fetch current docs for APIs
Context degradation	Summary of summary loses signal	Write to files, use fresh sessions, put rules in CLAUDE.md
Over-eager refactor	Model prefers "clean" over "minimal"	"Make the minimum change. Do not refactor surrounding code."

Failure Mode	Root Cause	Fix
Scope creep	Model runs commands you didn't ask for	Use permission deny/ask rules
Stale API knowledge	Training data cutoff	Always WebFetch current documentation
Niche language hallucination	Insufficient training data	Provide extensive examples and docs in context
False completion	Says "Done!" without verifying	CLAUDE.md: "Never declare complete without running tests"

---

12. Leverage the Full Extension System

Custom slash commands

`.claude/commands/test.md` → `/test MyComponent` for repeatable workflows. Keep commands simple — don't replace good CLAUDE.md with complex command libraries.

Skills (on-demand knowledge)

Load only when relevant, saving context. Use for specialized knowledge not needed every session.

MCP servers

GitHub, Jira, Slack, Notion, Figma, databases. Powerful but each adds token cost to every session.

Hooks

Deterministic Pre/PostToolUse actions: auto-formatting, linting, security gates, logging.

GitHub App

`/install-github-app` for automatic PR reviews. Valuable as AI-assisted coding increases PR volume.

---

13. Cost Management and Token Discipline

- **Average cost:** ~$6/developer/day, 90% under $12/day
- **Clear between tasks** — single easiest cost saving
- **Right model for the job** — Sonnet for routine, Opus for complex, Haiku for simple
- **Control thinking** — use `effort` parameter on Opus 4.6, lower budget for simple tasks
- **Lean CLAUDE.md** — every line reprocessed per message
- **Preprocess with hooks** — grep logs before letting Claude read them

- **Set automation safeguards** — `max_turns` and `timeout_minutes` for CI/CD

---

14. Working With Your Team

Version-control your configuration

- `CLAUDE.md` → committed, shared
- `.claude/settings.json` → committed (shared permissions, hooks)
- `.claude/settings.local.json` → git-ignored (personal preferences)
- `.claude/commands/` → committed (shared workflows)
- `.claude/agents/` → committed (shared subagent definitions)

Establish team conventions

- Which CLAUDE.md rules are mandatory
- Required hooks (formatting, linting)
- Permission baselines
- Model selection guidelines
- Session hygiene (when to `/clear`, when to `/compact`)

---

15. Security Considerations

Permission discipline

Define explicit allow/deny rules rather than blanket skip:

```json
{
  "permissions": {
    "allow": ["Read", "Edit", "Bash(npm run *)"],
    "deny": ["Bash(rm -rf *)", "Bash(sudo *)", "Bash(curl *)"],
    "ask": ["Bash(git commit *)", "Bash(git push *)"]
  }
}
```

Protect sensitive files

```json
```

```
{ "deny": ["Read(.env*)", "Read(./secrets/**)", "Write(production.config.*)"] }
```

Security review mandate for high-risk code

The model has seen both secure and insecure code in training and cannot reliably distinguish which is which. For auth, encryption, access control, and payment processing:

- Always run SAST tools (can be a PostToolUse hook)
- Mandate human security review
- Use the "devil's advocate" prompt specifically for security
- Never ship security-critical code without expert review, regardless of how confident the model sounds

---

16. The Meta-Skill: Think Like the Architecture

Everything in this guide follows from understanding how the system works:

Architecture Fact	Why It Matters	What To Do
Model is stateless	Knows nothing between sessions	Invest in CLAUDE.md
Context window is finite	Degrades when full	Manage context aggressively
Training data density varies	Better at Python than COBOL	Compensate weak areas with context
Agentic loop compounds errors	Each step risks failure	Plan first, minimize steps
RL trained for test passage	Optimized for verifiable code	Always run tests
LLMs are confident when wrong	Plausible hallucinations	Verify with tests, types, lint
Compaction loses information	Summaries degrade	Use fresh sessions, write to files
Probability follows strongest signal	Ambiguity → ambiguous output	Be specific, use types
Thinking tokens cost money	Extended thinking is expensive	Right-size thinking to task
Pattern strength varies by task	CRUD ≠ novel algorithms	Match oversight to task difficulty

The fundamental shift: You're no longer a developer who writes code. You're an **engineering lead who directs, constrains, and verifies an autonomous agent**. Your value is in knowing *what* to build, *how* it should be architected, *what quality bar* to hold, and *when to trust vs. verify* the agent's work.

Your job is to be the bridge between statistical generation and correct software. The model excels at pattern-rich, well-documented, verifiable tasks. You excel at novel thinking, architectural judgment, domain knowledge,

and quality assurance. The best results come from the combination — playing to each side's strengths.

The code is generated. The engineering is still yours.